

# Product Listing Generator

## Setup & Deployment Guide

A fully automated webshop listing tool: scrapes manufacturer pages, writes descriptions with Claude AI, generates spec tables for Shopify and Mailchimp, and produces transparent-background product images.

|                  |                                                                               |
|------------------|-------------------------------------------------------------------------------|
| <b>Input</b>     | A manufacturer product page URL                                               |
| <b>Output</b>    | description.md • specs-shopify.html • specs-mailchimp.html • transparent PNGs |
| <b>Hosting</b>   | Railway (Docker) — auto-deploys on every git push                             |
| <b>AI</b>        | Anthropic Claude claude-sonnet-4-5 via API                                    |
| <b>Alt. mode</b> | GitHub Actions workflow — no server required                                  |

## 1. Prerequisites

- Python 3.11 installed locally
- Git installed locally
- A GitHub account at [github.com](https://github.com)
- A Railway account at [railway.app](https://railway.app) — sign up with GitHub
- An Anthropic API key from [console.anthropic.com](https://console.anthropic.com)

The Anthropic API key is the only credential that costs money to use. Each product listing consumes approximately \$0.02–0.05 of API credit. Railway hosting runs approximately \$1–2/month for light use.

## 2. Project Structure

```

stripper-dipper/
■■■ main.py                # CLI entry point
■■■ pipeline.py            # Async orchestrator (shared by CLI and web)
■■■ scraper.py             # Playwright-based page scraper
■■■ generator.py           # Anthropic API: description + spec tables
■■■ image_processor.py     # rembg background removal + Pillow padding
■■■ app.py                 # FastAPI web server with SSE streaming
■■■ static/
■   ■■■ index.html        # Single-page web UI
■■■ requirements.txt
■■■ Dockerfile
■■■ .dockerignore
■■■ .gitignore
■■■ .github/
    ■■■ workflows/
        ■■■ generate-listing.yml    # GitHub Actions alternative workflow

```

### 3. Dependencies

Create **requirements.txt** with the following:

```

anthropic>=0.40.0
playwright>=1.44.0
rembg[cpu]>=2.0.57
Pillow>=10.0.0
fastapi>=0.110.0
uvicorn>=0.28.0
pydantic>=2.0.0

```

Install dependencies and download the Playwright Chromium browser:

```

pip install -r requirements.txt
playwright install --with-deps chromium

```

Note: rembg[cpu] installs onnxruntime for CPU-based inference. The U2Net background removal model (~170 MB) downloads automatically on first use.

### 4. Python Modules

#### scraper.py

- Launches a headless Chromium browser via Playwright.
- Loads the page with `wait_until='load'`, with a `domcontentloaded` fallback for sites that never fully settle.
- Extracts product name from `h1`, `[class*='product-title']`, `[itemprop='name']`, and similar selectors.
- Extracts description text from common description/overview container selectors.
- Extracts specifications from definition lists, two-column HTML tables, and class-hinted list items with colons.
- Extracts images by first searching gallery containers (`[class*='product-gallery']`, `swiper-slide`, `carousel-item`, etc.), then falling back to all page images filtered to `>150px` dimensions.

## generator.py

- `generate_description(product_data)` calls Claude `claude-sonnet-4-5` with a detailed system prompt.
- The prompt enforces: American English, product name in the first sentence, 1-3 paragraphs, no em dashes, no hollow hi-fi clichés (bans 'experience', 'sonic journey', 'immersive soundscape', etc.), and a conversational tone like a knowledgeable shop friend.
- Returns a tuple: (description, meta\_description). The meta description is SEO-optimised and capped at 150 characters.
- `generate_spec_tables(product_data)` generates two HTML tables in one API call using `===SHOPIFY===` and `===MAILCHIMP===` delimiters.
- Shopify table: standard HTML with minimal inline styling. Mailchimp table: fully inline CSS, safe email fonts, alternating row colors.
- Falls back to asking Claude to extract specs from raw page text if no structured specs were found.

## image\_processor.py

- Downloads each image with a browser-like User-Agent header.
- Downscales images to a maximum of 1200px on the longest side before processing — reduces memory use by ~6x with no visible quality loss.
- Removes the background using `rembg` (U2Net model).
- Pads the result to a square with a fully transparent background (RGBA).
- Saves as PNG with transparency preserved. Filenames use a slugified product name plus a sequential number (e.g. `hifiman-arya-01.png`).
- Calls `gc.collect()` after each image to release memory promptly.

## pipeline.py

- Async orchestrator shared by both the CLI and web server.
- Runs the Playwright scrape as a native async coroutine.
- Runs `generate_description` and `generate_spec_tables` via `asyncio.to_thread()` since the Anthropic SDK is synchronous.
- Runs `process_images` via `asyncio.to_thread()`; uses `loop.call_soon_threadsafe()` for thread-safe progress logging back to the async event loop.
- Accepts a `max_images` parameter (default 5) to control memory use.
- Saves all output files: `description.md`, `specs-shopify.html`, `specs-mailchimp.html`, and `images/*.png`.

## main.py

- CLI entry point. Reads the URL from `argv[1]`, the `PRODUCT_URL` environment variable, or an interactive prompt.
- Calls `asyncio.run(run_pipeline(url, print))` and exits.

## app.py

- FastAPI web server. Reads PORT from the environment (Railway injects this); binds to 0.0.0.0.
- POST /run — validates the URL, creates an asyncio.Queue, starts the pipeline as a background task, returns a job\_id.
- GET /stream/{job\_id} — Server-Sent Events endpoint. Reads from the job queue and streams progress messages to the browser. Sends a keep-alive ping every 10 seconds so the connection stays open during slow image processing.
- GET /download/{slug} — builds a ZIP of the output folder on the fly and returns it as a file download.
- Mounts the output/ directory as /files/ static files so the browser can load processed images directly.
- Logs whether ANTHROPIC\_API\_KEY is set at startup to aid debugging.

## static/index.html

- Single-page UI — no build step, no framework. Vanilla HTML, Tailwind CSS (CDN), and plain JavaScript.
- URL input field, max-images dropdown (1/2/3/5/8, default 5), and a Generate button.
- Live progress log rendered in a dark terminal-style panel via EventSource (SSE).
- Results section: description with Copy button; Google meta description with character counter and Copy button; spec table tabs (Shopify preview / Mailchimp preview / HTML source) each with a Copy HTML button; image gallery with transparent-background thumbnails (click to download); Download ZIP link.

## 5. GitHub Setup

### SSH key (one-time setup if not already done):

```
ssh-keygen -t ed25519 -C "github"  
cat ~/.ssh/id_ed25519.pub | pbcopy  
# Paste at: github.com/settings/ssh/new
```

### Initialise the repository and push:

```
git init  
git add .  
git commit -m "Initial commit"  
git remote add origin git@github.com:USERNAME/REPO.git  
git push -u origin main
```

### GitHub Actions workflow (.github/workflows/generate-listing.yml):

An alternative to the web UI — lets you run the pipeline entirely on GitHub's servers without any local setup or hosting costs.

- Trigger: workflow\_dispatch with a URL text input — run it from the GitHub Actions tab in the browser.
- Caches pip packages, Playwright's Chromium binary (~300 MB), and the rembg U2Net model (~170 MB) between runs.
- The ANTHROPIC\_API\_KEY is stored as a GitHub Actions secret (Settings > Secrets > Actions > New repository secret).
- Output is uploaded as a downloadable ZIP artifact, retained for 90 days.

## 6. Dockerfile

---

Docker lets Railway (or any container host) reproduce the environment exactly, including Playwright's system-level Chromium dependencies.

```
FROM python:3.11-slim

WORKDIR /app

# Install Python dependencies first for better layer caching
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Install Playwright Chromium and all required system libraries
RUN playwright install --with-deps chromium

COPY . .

# Output directory must exist for the static file mount at startup
RUN mkdir -p output

EXPOSE 8000
CMD ["python", "app.py"]
```

Also create a .dockerignore file to keep the image lean:

```
.git
.github
output/
__pycache__/
*.pyc
.env
.DS_Store
```

## 7. Railway Deployment

---

### 1. Sign in to Railway

Go to [railway.app](https://railway.app) and click Login with GitHub.

### 2. Create a new project

Click New Project > Deploy from GitHub repo, then select your repository.

### 3. Wait for the build

Railway detects the Dockerfile automatically and begins building. The first build takes 3-5 minutes as it installs Chromium and all dependencies.

### 4. Add the API key

In the Railway dashboard, click your service > Variables tab > New Variable. Set name to ANTHROPIC\_API\_KEY and paste your key from [console.anthropic.com](https://console.anthropic.com).

### 5. Generate a domain

Click Settings tab > Networking > Generate Domain. Railway assigns a free \*.up.railway.app URL.

## 6. Verify it is running

Open the URL in a browser. The Product Listing Generator UI should appear.

## 7. Automatic deploys

Every subsequent git push to the main branch triggers a new Railway deployment automatically.

| Service         | Cost                                                |
|-----------------|-----------------------------------------------------|
| Railway hosting | ~\$1–2/month for light use                          |
| Anthropic API   | ~\$0.02–0.05 per product listing                    |
| GitHub          | Free (2,000 Actions minutes/month on private repos) |

## 8. Using the App

### Web UI (Railway)

- Open your \*.up.railway.app URL in any browser.
- Paste a manufacturer product page URL into the input field.
- Choose how many images to process from the dropdown (recommend 3-5; fewer = faster and less memory).
- Click Generate listing and watch the live progress log.
- When complete, the results section appears: copy the description, meta description, or spec table HTML with one click. Download images individually or as a ZIP.

### GitHub Actions (no server required)

- Go to your GitHub repo > Actions tab > Generate Product Listing.
- Click Run workflow, paste the product URL, click Run workflow.
- When the run completes (green checkmark), scroll to Artifacts and download the ZIP.

### CLI (local)

```
python main.py https://example.com/product-page
```

## 9. Output Structure

```
output/[product-name-slug]/
■■■ description.md          # Product description + Google meta description
■■■ specs-shopify.html     # Spec table formatted for Shopify
■■■ specs-mailchimp.html   # Spec table with fully inline CSS for email
■■■ images/
    ■■■ product-name-01.png # Transparent background PNG
    ■■■ product-name-02.png
    ■■■ ...
```

Images are named using a slugified version of the product name and a two-digit sequence number. All images are RGBA PNGs with transparent backgrounds, square-padded with 20px of clear space on each side.